



دانشکده علوم ریاضی

رشته علوم کامپیوتر

گزارش تمرین 4

محاسبه Lagrange-Interpolation با استفاده از

Serial و مقایسه اش با روش CUDA

تهیه کننده:

ماریا صبوری دودران

شماره دانشجویی: 402222001

مردادماه

فهرست

3	 توضیح کد
3	:Serial
3	 نمودار ها
6	:CUDA
6	lagrangeGPU
7	 Mean_error_calc
7	Shared_mem_sum
8	Main
8	 نمودار ها
11	 آزمایش کارایی
12	block بررسی اثر اندازه

نکته: لطفا کد از روی گیت هاب ام چک شود.¹

¹ <https://github.com/MariaSabouri/CUDA-Lagrange-Interpolation.git>

توضیح کد

مطابق خواسته صورت سوال، قرار است که روش lagrange در یه محاسبه Heterogeneous را با محاسبه Serial و در CPU را بررسی کرد.

Serial:

در این کد، 3 لوپ وجود دارد، لوپ بیرونی مربوط به نقاط پرسوجو و لوپ میانی بهش جمع رابطه لاگرانژ و لوپ داخلی مربوط به محاسبه پایه لاگرانژ است.

در خط 32، درجا جمع خطای کلی نقاط پرسوجو حساب شده و در خارج 3 لوپ میانگین خطا ها از مقدار اصلی حساب شده.

نکته ای که اینجا مهم بود توجه کنم این هست که برای مقایسه عملکرد CUDA با روش Serial، همه متغیرها رو float تعریف کردم. به عبارتی اینطور نبوده که یک متغیر double و متغیر دیگه float باشه. در غیر اینصورت، خطاهای مربوط به گرد کردن این دو متغیر میتواند تفاوت زیادی در خروجی serial و CUDA داشته باشد. (این مورد رو قبلا نمیدونستم و وقتی بررسی کردم متوجه شدم.)

```
1 void lagrange(
2     const float a,
3     const int n,
4     const int query_n,
5     const float h,
6     const float query_h,
7     float* result,
8     float& mean_lag_error
9 )
10 {
11     for(int q = 0; q < query_n; ++q)
12     {
13         result[q] = 0.0f;
14         float x_value = a + q * query_h;
15         for (int i = 0; i < n; ++i)
16         {
17             float basis = 1.0f;
18             float x_i = a + i * h;
19
20             for (int j = 0; j < n; ++j)
21             {
22                 if (i != j)
23                 {
24                     float x_j = a + j * h;
25                     basis *= (x_value - x_j) / (x_i - x_j);
26                 }
27             }
28
29             float y_i = U(x_i);
30             result[q] += y_i * basis;
31         }
32         mean_lag_error += fabsf(result[q] - U(x_value));
33     }
34     mean_lag_error /= query_n;
35 }
```

نمودار ها

خروجی حاصل چنین است:

slice_number	block_size	query_num	Time Elapsed(us)	mean_error	version
8	0	10000	2.09E+03	5.54E-06	Serial
8	0	100000	2.76E+04	5.55E-06	Serial
8	0	1000000	2.34E+05	5.54E-06	Serial
16	0	10000	7.96E+03	1.59E-06	Serial
16	0	100000	8.17E+04	1.55E-06	Serial
16	0	1000000	7.78E+05	1.55E-06	Serial
32	0	10000	2.82E+04	2.17E-02	Serial
32	0	100000	3.03E+05	2.15E-02	Serial
32	0	1000000	2.97E+06	2.22E-02	Serial
64	0	10000	1.32E+05	1.83E+07	Serial

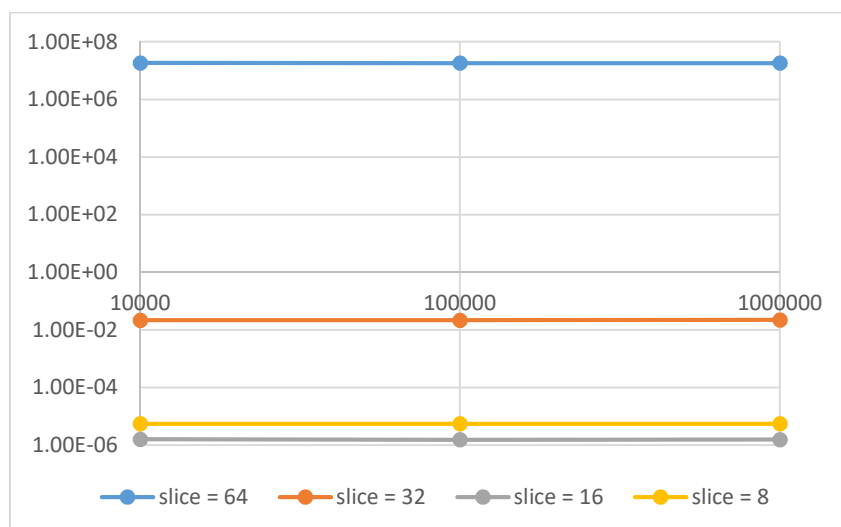
64	0	100000	1.24E+06	1.81E+07	Serial
----	---	--------	----------	----------	--------

که نتایج به صورت پلات شده مطابق شکل زیر است:

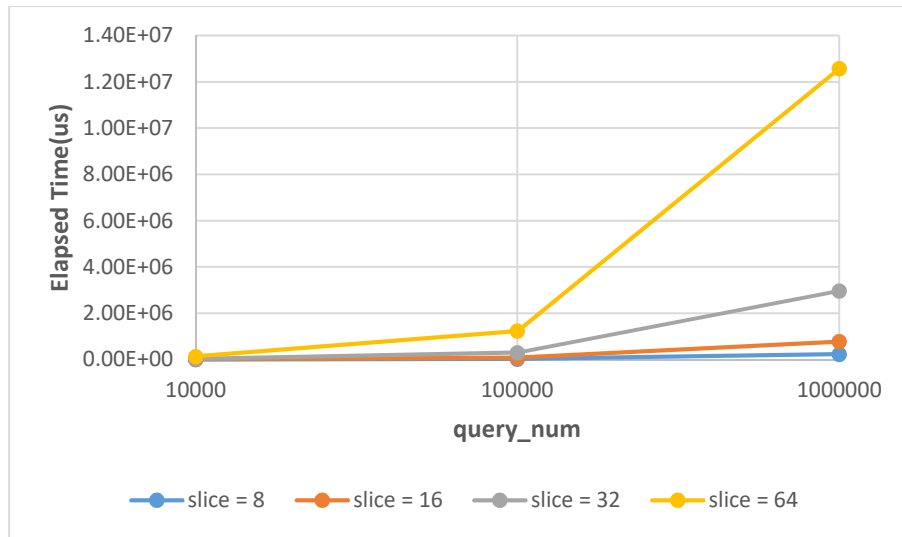


که slice تعداد نقاط داده و query_num تعداد نقاط پرسوجو است. mean_error هم میانگین قدرمطلق خطای نقطه پرسوجو از مقدار واقعی است.

نمودار کلی به شکل زیر است:



مهم ترین نمودار مربوط به زمان صرف شده برای محاسبه به صورت سریال است که طبق نمودار زیر است:



CUDA:

در بخش کد در CUDA، من 2 تا تابع kernel تعریف کردم. یک تابع مخصوص محاسبه درون یابی لاگرانژ و تابع دیگر مخصوص محاسبه میانگین خطا به ازای هر slice_num و روی queru_num تا نقطه ی پرسوجو است.

lagrangeGPU

در کرنل lagrangeGPU، منطق کد این است:

```
1  __global__ void lagrangeGPU(
2      const float a,
3      const float h,
4      const float query_h,
5      const int n,
6      float* lag,
7      float* mean_lag_error_arr
8  )
9  {
10     __shared__ float thread_calcs[MAX_BLK_SZ];
11     __shared__ float warp_sum_arr[WARP_SZ];
12
13     int query_i = blockIdx.x;
14     int my_i = threadIdx.x;
15     int my_warp = threadIdx.x / warpSize;
16     int my_lane = threadIdx.x % warpSize;
17
18     float* shared_vals = thread_calcs + my_warp * warpSize;
19     float blk_result = 0.0f;
20
21     shared_vals[my_lane] = 0.0f;
22
23     if (my_i < n)
24     {
25         float my_x = a + my_i * h;
26         float li = 1.0f;
27
28         for (int k = 0; k < n; k++)
29         {
30             if (k != my_i)
31             {
32                 float x_k = a + k * h;
33                 float x_input = a + query_i * query_h;
34
35                 li *= (x_input - x_k) / (my_x - x_k);
36             }
37         }
38
39         float my_y = U(my_x);
40         shared_vals[my_lane] = my_y * li;
41     }
42
43     float my_result = Shared_mem_sum(shared_vals);
44     if(my_lane == 0) warp_sum_arr[my_warp] = my_result;
45     __syncthreads();
46
47     if(my_warp == 0)
48     {
49         if(threadIdx.x >= (blockDim.x + warpSize - 1) / warpSize)
50             warp_sum_arr[threadIdx.x] = 0.0;
51         blk_result = Shared_mem_sum(warp_sum_arr);
52     }
53
54     if(threadIdx.x == 0){
55         lag[query_i] = blk_result;
56         float x = a + query_i * query_h;
57         mean_lag_error_arr[query_i] += fabs(U(x) - blk_result);
58     }
59 }
```

- 1- برای هر نقطه پرسوجو یک block در نظر گرفته بودم. که خط 13 عکس روبرو نشان دهنده تعریف متغیر query_num بر این اساس است.
- 2- برای هر سیگمای رابطی لاگرانژ یک tread در نظر گرفتم که خط 14 نشان دهنده ی آن است. هر thread در هر block مسئول این هست که پایهی لاگرانژ خودش را بسازد و در آرایهی thread_calcs thread_calcs متغیر shared هست بین thread های هر warp در یک block.
- 3- اینکه در کدام warp هستیم و هر thread در کدام موقعیت نسبت به warp قرار دارد در خط های 15 و 16 تعریف کردم.
- 4- در خط 43، هر آرایهی thread_calcs در warp توسط تابع Shared_mem_sum جمع میشوند و حاصل هر warp در آرایهی warp_sum_arr ذخیره میشود. توجه که همچنان این آرایه هم shared است؛ یعنی بین thread های هر block مشترک است. توجه که فقط lane = 0 در هر warp نماینده این هست که در اندیس my_warp این آرایه حاصل warp اش را بگذارد.
- 5- با دستور __syncthreads هم مطمئن میشویم که همه ی warp ها کارشون را تمام کردند.
- 6- با توجه به اینکه هر block حداقل یک warp دارد، من تصمیم گرفتم که در warp = 0 حاصل جمع warp_sum_arr را انجام دهم. خط 49 به این دلیل است که:

لزوما همه 32 thread هر warp در

- warp_sum_arr متغیردهی نشدند؛ زیرا لزوما هر block را 1024 thread در نظر نگرفتیم و اندازه ی block میتواند کوچک تر باشد. خلاصه اینکه نتیجه ی این آرایه هم در blk_result ذخیره میشود.
- 7- خط 54 تا 58 هم به این دلیل هست که thread = 0 هر block را مسئول این میدونیم که بیاید در متغیرهای global به اسم lag و mean_lag_error_arr مقادیر نهایی را وارد کند.

Mean_error_calc

```
1 __global__ void mean_error_calc(
2     const int n,
3     float* mean_lag_error_arr,
4     float* mean_lag_error
5 )
6 {
7     __shared__ float thread_calcs[MAX_BKLSZ];
8     __shared__ float warp_sum_arr[WARP_SZ];
9
10    int my_i = blockIdx.x * blockDim.x + threadIdx.x;
11    int my_warp = threadIdx.x / warpSize;
12    int my_lane = threadIdx.x % warpSize;
13
14    float* shared_vals = thread_calcs + my_warp * warpSize;
15    float blk_result = 0.0f;
16
17    shared_vals[my_lane] = 0.0f;
18    if (my_i < n)
19    {
20        shared_vals[my_lane] = mean_lag_error_arr[my_i];
21    }
22    float my_result = Shared_mem_sum(shared_vals);
23    if(my_lane == 0) warp_sum_arr[my_warp] = my_result;
24    __syncthreads();
25
26    if(my_warp == 0)
27    {
28        if(threadIdx.x >= (blockDim.x + warpSize - 1) / warpSize)
29            warp_sum_arr[threadIdx.x] = 0.0;
30        blk_result = Shared_mem_sum(warp_sum_arr);
31    }
32    if(threadIdx.x == 0){
33        atomicAdd(mean_lag_error, blk_result);
34    }
35 }
```

بعد از اتمام کار kernel اول، نوبت به محاسبه میانگین خطای نقاط پرسوجو است.

منطق کد شباهت زیادی با کرنل قبل دارد؛ با این تفاوت که بر خلاف قبلی باید نتایج همه‌ی block ها را با هم در یک متغیر global جمع کنیم.

Shared_mem_sum

```
1 __device__ float Shared_mem_sum(float shared_vals[])
2 {
3     int my_lane = threadIdx.x % warpSize;
4
5     for(int diff = warpSize / 2; diff > 0; diff = diff / 2)
6     {
7         if (my_lane < diff)
8         {
9             shared_vals[my_lane] += shared_vals[my_lane + diff];
10        }
11        __syncwarp();
12    }
13    return shared_vals[0];
14 }
```

Dissemination sum using همان کد به
shared memory است و با یک مقدار تفاوت. به
عبارتی با توجه به اینکه یک نیمه از thread های هر
warp در هر لوپ باید متغیر دهی بشوند و بقیه‌ی
my_lane > diff نیاز به متغیر دهی نیستند. خط 7 به
همین دلیل آورده شده است.

Main

این بخش، در CPU انجام میشود و من متغیردهی های اولیه رادر اینجا انجام دادم. با توجه به اینکه capability مورد استفاده از 2 بیشتر است، از cudaMallocManaged استفاده کردم.

The screenshot shows a Jupyter Notebook with a terminal window displaying the output of a CUDA program. The output lists performance metrics for various configurations of slice number, block size, and query number. The metrics include Compute Capability, CUDA kernel time, Mean absolute error, and GPU time. The configurations are as follows:

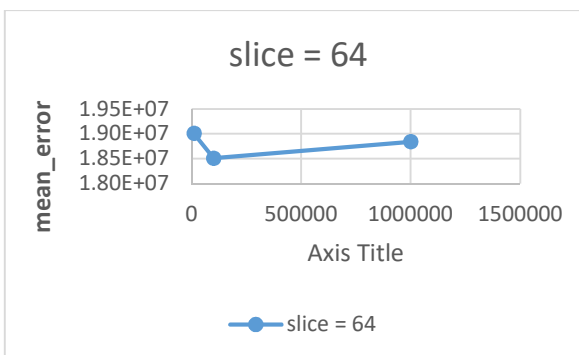
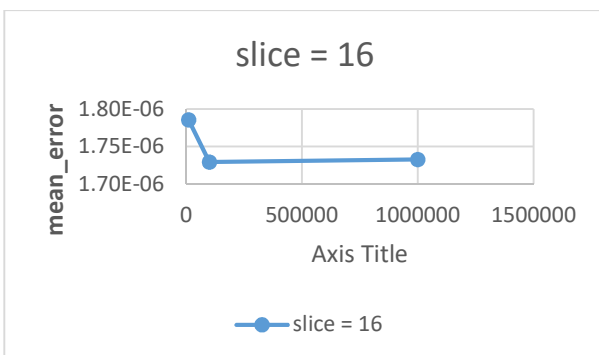
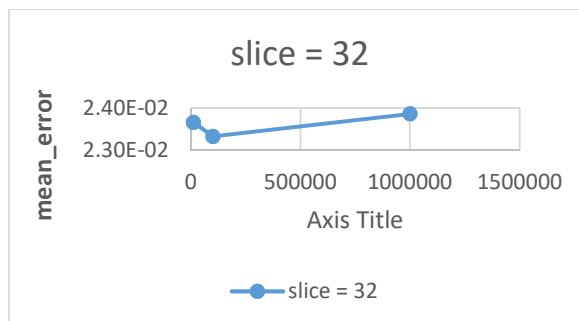
slice_num	block_size	query_num	Time Elapsed(us)	mean_error	version
8	32	10000	5.53E+02	5.55E-06	CUDA
8	32	100000	1.40E+03	5.55E-06	CUDA
8	32	1000000	9.08E+03	5.55E-06	CUDA
16	32	10000	5.19E+02	1.79E-06	CUDA
16	32	100000	1.34E+03	1.73E-06	CUDA
16	32	1000000	1.34E+04	1.73E-06	CUDA
32	32	10000	5.80E+02	2.37E-02	CUDA
32	32	100000	2.13E+03	2.33E-02	CUDA
32	32	1000000	1.32E+04	2.39E-02	CUDA
64	64	10000	8.87E+02	1.90E+07	CUDA
64	64	100000	5.18E+03	1.85E+07	CUDA
64	64	1000000	3.55E+04	1.88E+07	CUDA

نمودار ها

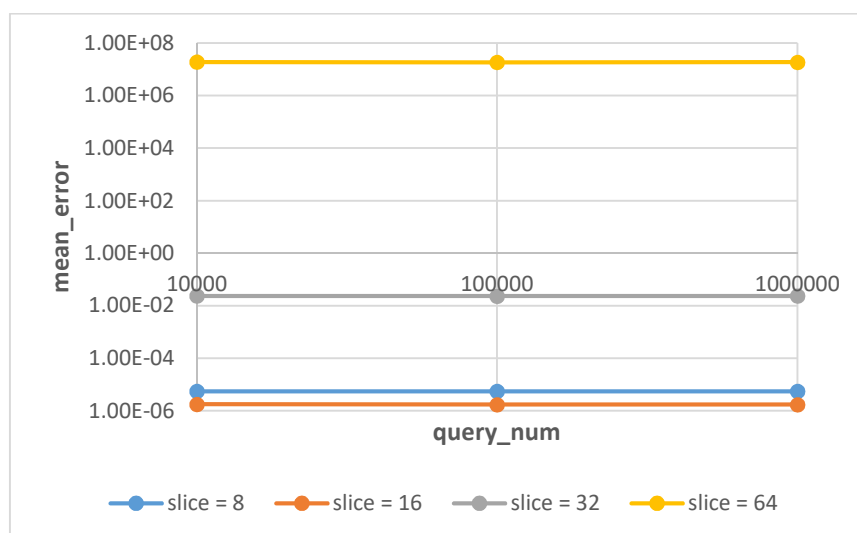
خلاصه نتایج مطابق جدول زیر است:

slice_num	block_size	query_num	Time Elapsed(us)	mean_error	version
8	32	10000	5.53E+02	5.55E-06	CUDA
8	32	100000	1.40E+03	5.55E-06	CUDA
8	32	1000000	9.08E+03	5.55E-06	CUDA
16	32	10000	5.19E+02	1.79E-06	CUDA
16	32	100000	1.34E+03	1.73E-06	CUDA
16	32	1000000	1.34E+04	1.73E-06	CUDA
32	32	10000	5.80E+02	2.37E-02	CUDA
32	32	100000	2.13E+03	2.33E-02	CUDA
32	32	1000000	1.32E+04	2.39E-02	CUDA
64	64	10000	8.87E+02	1.90E+07	CUDA
64	64	100000	5.18E+03	1.85E+07	CUDA
64	64	1000000	3.55E+04	1.88E+07	CUDA

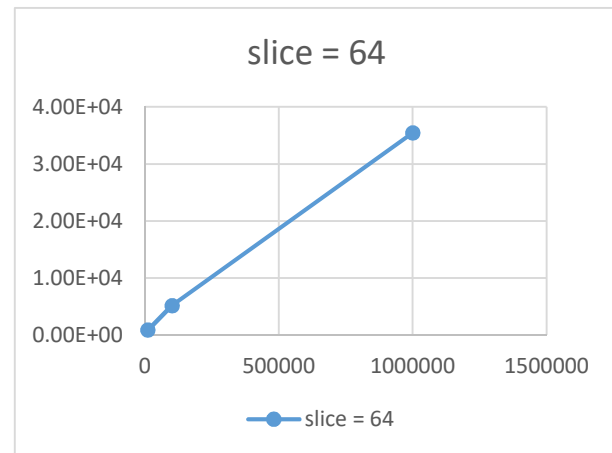
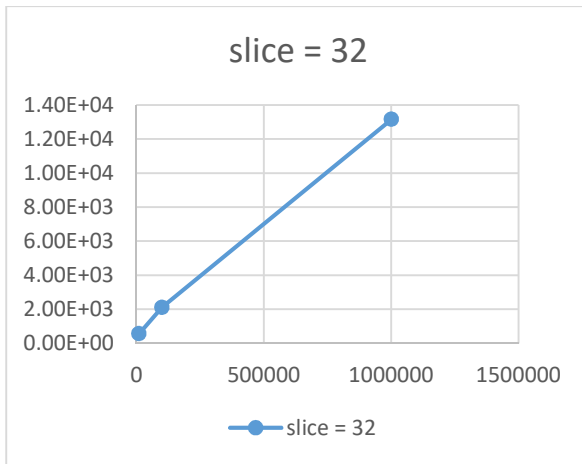
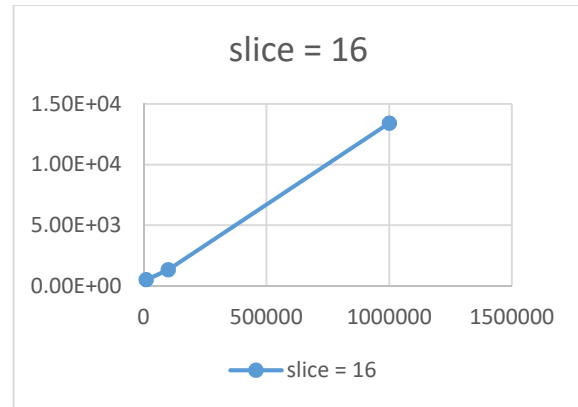
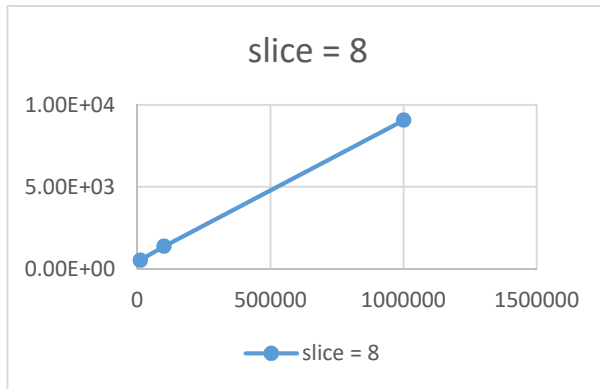
که نتایج میانگین خطاها به صورت پلات شده مطابق شکل زیر است:



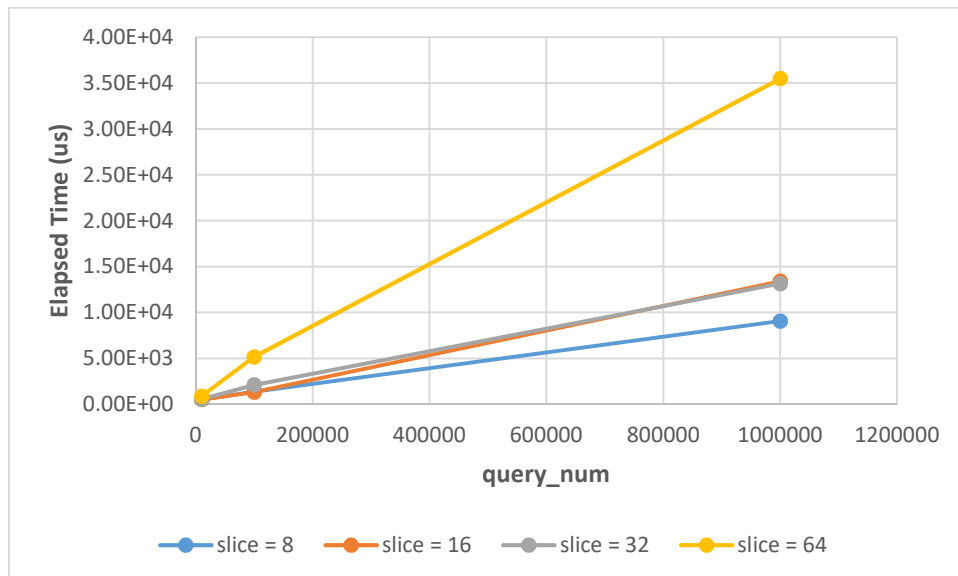
نمودار کلی به شکل زیر است:



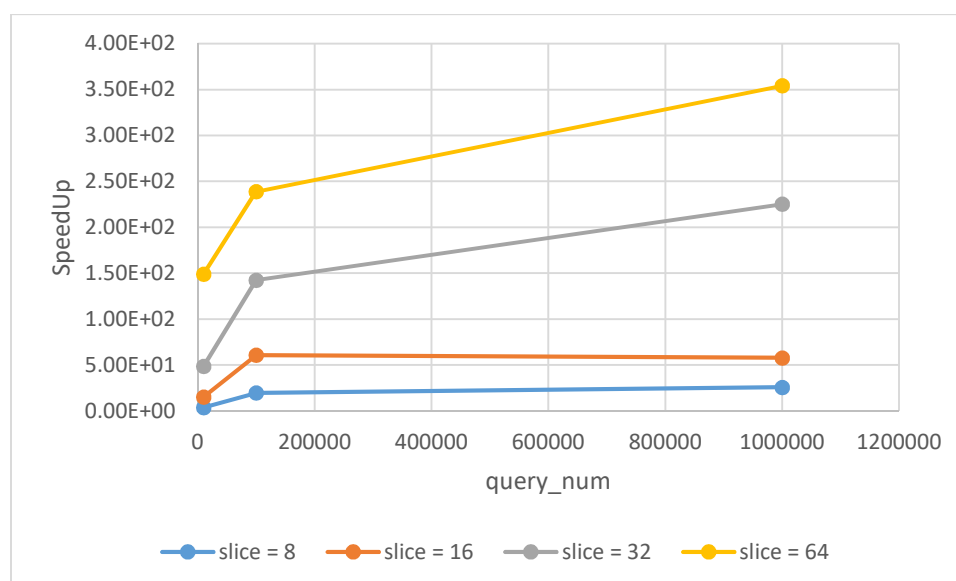
حال نمودار زمان سپری شده برای هر slice_num مطابق زیر است:



که نمودار ها به صورت کلی با هم به شکل زیر است:



slice_number	query_num	Time Elapsed(us)-CUDA	Time Elapsed(us) - serial	SpeedUP
8	10000	5.53E+02	2.09E+03	3.77E+00
8	100000	1.40E+03	2.76E+04	1.97E+01
8	1000000	9.08E+03	2.34E+05	2.57E+01
16	10000	5.19E+02	7.96E+03	1.53E+01
16	100000	1.34E+03	8.17E+04	6.08E+01
16	1000000	1.34E+04	7.78E+05	5.79E+01
32	10000	5.80E+02	2.82E+04	4.87E+01
32	100000	2.13E+03	3.03E+05	1.43E+02
32	1000000	1.32E+04	2.97E+06	2.25E+02
64	10000	8.87E+02	1.32E+05	1.49E+02
64	100000	5.18E+03	1.24E+06	2.39E+02
64	1000000	3.55E+04	1.26E+07	3.54E+02



نمودار نشان می‌دهند که نسخه CUDA در تمامی حالت‌های آزمایش‌شده سریع‌تر از نسخه CPU است. همچنین با افزایش query_num، مقدار Speedup افزایش پیدا می‌کند، زیرا با افزایش تعداد نقاط پرس‌وجو، حجم محاسبات بیشتر شده و overhead اجرای GPU نسبت به کل زمان محاسبه سهم کمتری پیدا می‌کند. برای مثال، در slice_num = 8، مقدار Speedup از 3.77 برای 10,000 نقطه به 25.7 برای 1,000,000 نقطه افزایش پیدا کرده است.

در حالت‌هایی که اندازه مسئله کوچک است، سربارهایی مانند راه‌اندازی kernel، مدیریت thread ها و مدیریت حافظه سهم بیشتری از زمان اجرا را به خود اختصاص می‌دهند و در نتیجه مزیت استفاده از GPU کمتر دیده می‌شود. اما با افزایش اندازه مسئله، این سربارها در مقایسه با حجم محاسبات ناچیزتر می‌شوند و توان پردازش موازی GPU بیشتر مورد استفاده قرار می‌گیرد. بنابراین می‌توان گفت که با افزایش query_num و slice_num و در نتیجه افزایش حجم محاسبات، مزیت نسخه CUDA نسبت به CPU بیشتر می‌شود.

بررسی اثر اندازه block

تغییر اندازه block تأثیر قابل توجهی بر زمان اجرای نسخه CUDA دارد. به عبارتی با افزایش block_size از 64 به 128، 256 و 512، زمان اجرای برنامه در بیشتر حالت‌ها افزایش پیدا کرده است. اندازه block_size = 64 در بیشتر حالت‌های آزمایش بهترین زمان اجرا را دارد. افزایش تعداد thread های هر block لزوماً باعث افزایش سرعت نمی‌شود، زیرا با افزایش اندازه block، تعداد thread های بیشتری به صورت همزمان منابع GPU مانند register و shared memory را مصرف می‌کنند و در نتیجه ممکن است تعداد block های قابل اجرای همزمان روی SM کاهش پیدا کند. افزایش تعداد warp های موجود در هر block می‌تواند باعث افزایش مصرف منابع و کاهش تعداد block های قابل اجرای همزمان شود. انتخاب اندازه block مناسب به تعادل بین میزان parallelisation و میزان منابع مورد استفاده بستگی دارد.

